

Contribution Technique

Table of Contents

1. The Fundamental Contribution Principle
2. Single-Element Contribution
3. Pair Contribution
4. Boundary-Counting Contribution
5. Maximum / Minimum Contribution (Monotonic Stack Contribution)
6. Distinct-Element Contribution
7. Bitwise Contribution (XOR / AND / OR)
8. Prefix-Sum Contribution (Prefix-Pair Contribution)
9. Median Contribution (Balance Transformation)
10. GCD / LCM Contribution
11. Frequency-Based Contribution (Occurrence Contribution)
12. Structural Contribution (Tree / Graph Contribution)

1. The Fundamental Contribution Principle

The Fundamental Contribution Principle is the idea that instead of directly computing a complicated global expression, we rewrite the problem by asking a simpler question: how much does each element contribute to the final answer across all valid structures (like subarrays, pairs, or ranges)?

This idea becomes very powerful when the final answer is written as a double summation over all ranges. For example, many problems start from something like:

$$\sum_{1 \leq l \leq r \leq n} \left(\sum_{i=l}^r a_i \right)$$

This expression means: take every subarray $[l, r]$, compute its sum $\sum_{i=l}^r a_i$, and then add all those subarray sums together.

🔄 Step 1: Flip the viewpoint

Instead of summing over subarrays first, we reverse the order of thinking. We ask: for a fixed index i , in how many subarrays does a_i appear?

So we rewrite the idea as:

$$\sum_{i=1}^n a_i \cdot (\text{number of subarrays that include } i)$$

Now the problem becomes much easier because we no longer deal with all subarrays directly.

• Step 2: Count contribution of one element

A subarray $[l, r]$ contains index i if:

$$l \leq i \leq r$$

So:

- l can be chosen from 1 to i
- r can be chosen from i to n

Number of choices:

$$\text{left choices} = i \quad \text{right choices} = (n - i + 1)$$

So total subarrays containing i is:

$$i \cdot (n - i + 1)$$

★ Step 3: Final simplified form

Now substitute this back:

$$\sum_{i=1}^n a_i \cdot i \cdot (n - i + 1)$$

⊙ Final Idea (Core Principle)

The whole principle can be summarized as:

Instead of computing:

$$\sum_{\text{all structures}} \text{value}(\text{structure})$$

We rewrite it as:

$$\sum_{i=1}^n \text{contribution of } a_i$$

where:

$$\text{contribution of } a_i = a_i \times (\text{number of structures containing } i)$$

2. Single-Element Contribution

The Single-Element Contribution technique is based on a simple idea: instead of analyzing all subarrays at once, we focus on one element a_i and calculate how many times it contributes to the final answer across all valid subarrays.

Suppose we start from the standard expression of sum over all subarrays:

$$\sum_{0 \leq l \leq r < n} \left(\sum_{i=l}^r a_i \right)$$

Instead of expanding this directly, we fix a single index i and ask: in how many subarrays does a_i appear?

A subarray $[l, r]$ contains index i if:

$$l \leq i \leq r$$

Now we count the choices using 0-based indexing:

- The left endpoint l can be chosen from 0 to i , so there are $(i + 1)$ choices
- The right endpoint r can be chosen from i to $n - 1$, so there are $(n - i)$ choices

So the number of subarrays that contain index i is:

$$(i + 1)(n - i)$$

Therefore, the total contribution of element a_i is:

$$a_i \cdot (i + 1)(n - i)$$

Now instead of summing over all subarrays, we sum over all elements:

$$\sum_{i=0}^{n-1} a_i \cdot (i + 1)(n - i)$$

This is the key idea of single-element contribution: each element contributes independently based on how many subarrays include its index.

Problem Types Related to This Concept

This technique is commonly used in problems such as:

- Sum of all subarray sums
- Total contribution of array elements over all subarrays
- Weighted subarray sum problems
- Problems where each element contributes based on its position in a range rather than the range itself

3. Pair Contribution

Pair contribution is the natural extension of single-element contribution. Instead of asking “how many subarrays contain index i ?”, we ask “how many subarrays contain both indices i and j ?”

For $i < j$, a subarray $[l, r]$ contains both indices if and only if:

$$l \leq i \quad \text{and} \quad r \geq j$$

So the left endpoint can be chosen in $i + 1$ ways and the right endpoint can be chosen in $n - j$ ways. Since these choices are independent, the number of subarrays containing both i and j is:

$$(i + 1)(n - j)$$

Starting Point: Double Sum Structure

Many problems naturally lead to expressions like:

$$\sum_{0 \leq i < j < n} a_i a_j$$

This means we are summing over all pairs (i, j) , where each pair contributes $a_i a_j$.

Instead of thinking in terms of pairs directly, we convert everything into algebraic expansions of full sums.

Core Identity: Square of a Sum

We start from a fundamental algebra rule:

$$\left(\sum_{i=0}^{n-1} a_i \right)^2$$

Expanding this gives:

$$\left(\sum a_i \right)^2 = \sum_{i=0}^{n-1} a_i^2 + 2 \sum_{0 \leq i < j < n} a_i a_j$$

This is extremely important because it separates:

- self interaction: a_i^2
- pair interaction: $a_i a_j$

Now we isolate the pair part:

$$\sum_{0 \leq i < j < n} a_i a_j = \frac{1}{2} \left[\left(\sum_{i=0}^{n-1} a_i \right)^2 - \sum_{i=0}^{n-1} a_i^2 \right]$$

So instead of iterating over all pairs, we only need:

- sum of elements
- sum of squares

Full Double Sum Expansion Idea

A more general identity is:

$$\left(\sum_{i=0}^{n-1} a_i \right) \left(\sum_{j=0}^{n-1} b_j \right) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} a_i b_j$$

This means:

- every a_i pairs with every b_j

- this is a full cross interaction

If $a = b$, then:

$$\left(\sum a_i\right)^2 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} a_i a_j$$

Now we split it into two parts:

$$\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} a_i a_j = \sum_{i=j} a_i^2 + \sum_{i \neq j} a_i a_j$$

So:

$$\sum_{i \neq j} a_i a_j = \left(\sum a_i\right)^2 - \sum a_i^2$$

And unordered pairs become:

$$\sum_{0 \leq i < j < n} a_i a_j = \frac{(\sum a_i)^2 - \sum a_i^2}{2}$$

Key Insight Behind Pair Contribution

The main idea is:

Instead of directly counting pairs, we expand everything using algebra:

- $\sum a_i$ captures single contributions
- $(\sum a_i)^2$ automatically generates all pair interactions
- subtracting $\sum a_i^2$ removes self-pairs
- dividing by 2 removes double counting

So pair contribution is not combinatorics first—it is algebra first, counting later.

Distance Contribution Extension

The same philosophy works for distance-related problems. Suppose we want:

$$\sum_{\text{subarrays } i < j \in \text{subarray}} (j - i)$$

For every pair (i, j) , the distance $j - i$ contributes in every subarray containing both indices, so its total contribution is:

$$(j - i)(i + 1)(n - j)$$

The difficulty is the factor $j - i$, which is not separable. The standard trick is to expand it:

$$j - i = j + (-i)$$

Then:

$$\sum_{i < j} (j - i)(i + 1)(n - j)$$

becomes:

$$\sum_{i < j} j(i + 1)(n - j) - \sum_{i < j} i(i + 1)(n - j)$$

Now each term is separable and can be handled with the same running-prefix technique. Whenever you see expressions like $(r - l)$, $(j - i)$, or similar differences, always try splitting them into separate sums.

Higher Powers of Subarray Sums

The idea extends even further to higher powers of subarray sums. For squares, we get single-element terms and pair terms. For cubes:

$$\left(\sum a_i\right)^3 = \sum a_i^3 + 3 \sum_{i \neq j} a_i^2 a_j + 6 \sum_{i < j < k} a_i a_j a_k$$

Now triples appear. The key observation is that a subarray contains indices $i < j < k$ if and only if it contains the two extreme indices i and k . Once those are included, the middle index j is automatically included because subarrays are contiguous. Therefore the number of subarrays containing the entire triple is simply:

$$(i+1)(n-k)$$

Notice that the count depends only on the smallest and largest index, not on the middle index. This is why triple contributions can still be computed efficiently. We maintain running accumulators for single elements, then pair contributions, and then triple contributions. Each new accumulator is updated from the previous one, so a cubic expression still takes only $O(n)$ time rather than $O(n^3)$.

Final Intuition

Pair contribution is just this:

Instead of thinking:

- all pairs one by one

We think:

- $(\sum a_i)^2$ and then carefully subtract what we don't want.

4. Boundary-Counting Contribution

Boundary-counting contribution is one of the most important generalizations of the basic formula:

$$(i + 1)(n - i)$$

Previously, when counting how many subarrays contain index i , the only requirement was that the subarray must include i . So the left endpoint could be chosen from 0 to i , and the right endpoint from i to $n - 1$. This gives:

$$(i + 1)(n - i)$$

Adding Constraints (Why Boundary Appears)

However, many problems impose extra conditions. For example:

- $a[i]$ must be the minimum in the subarray
- $a[i]$ must be the maximum
- $a[i]$ is the first or last occurrence
- some condition that can be broken by nearby elements

In such cases, not every subarray containing i is valid. Some neighboring indices “break” the condition.

So instead of the full range $[0, n - 1]$, we are restricted to a smaller safe interval.

Nearest Breaking Boundaries

Suppose:

- L is the nearest index on the left that breaks the condition
- R is the nearest index on the right that breaks the condition

Then any valid subarray must stay strictly inside these boundaries.

So:

- left endpoint l can be chosen from $L + 1$ to i
- right endpoint r can be chosen from i to $R - 1$

Counting Valid Choices

Number of choices:

Left side: $i - L$

Right side: $R - i$

Since these choices are independent, total valid subarrays are:

$$(i - L)(R - i)$$

This is the boundary-counting formula.

Intuition (Wall Interpretation)

Think of L and R as walls:

- you cannot extend the subarray beyond these walls
- as long as the subarray stays inside, the property holds

So:

- left freedom = distance from L to i
- right freedom = distance from i to R

Multiplying them gives total valid subarrays.

Connection to Basic Formula

Notice something important:

$$(i+1)(n-i)$$

is just a special case of boundary counting. If there are no breaking points, we set:

$$L = -1, \quad R = n$$

Then:

$$(i - (-1))(n - i) = (i + 1)(n - i)$$

So the basic formula is just a special case of boundary contribution.

Classic Application: Subarray Minimums

In the sum of subarray minimums problem:

- find nearest smaller element on the left $\rightarrow L$
- find nearest smaller-or-equal element on the right $\rightarrow R$

Then every subarray where i is the minimum must lie inside (L, R) .

So contribution of $a[i]$ is:

$$a[i](i - L)(R - i)$$

Summing over all i gives the answer in $O(n)$ using a monotonic stack.

Subarray Maximums

For maximums:

- use nearest greater elements instead of smaller ones

Still the same formula:

$$a[i](i - L)(R - i)$$

Only the definition of boundaries changes.

Histogram Rectangle

In largest rectangle in histogram:

- each bar $a[i]$ is height
- boundaries are nearest smaller elements

Width becomes:

$$R - L - 1$$

So area is:

$$a[i](R - L - 1)$$

This is again the same boundary idea, just measured differently.

Distinct / Frequency Contribution

In distinct-element problems:

- previous occurrence acts as left boundary L
- next occurrence acts as right boundary R

We again get:

$$(i - L)(R - i)$$

This counts how many subarrays where this occurrence is the “first valid appearance” or uniquely responsible occurrence.

Final Insight

Boundary contribution is the universal counting structure behind most subarray problems.

Once the nearest breaking indices are found:

- using monotonic stack
- previous/next occurrence arrays

the entire problem collapses into:

$$(i - L)(R - i)$$

This single formula powers:

- subarray minimums
- subarray maximums
- histogram problems
- distinct-element counting
- frequency-based contributions

It is one of the most fundamental tools in contribution technique.

5. Maximum / Minimum Contribution (Monotonic Stack Contribution)

Maximum and minimum contribution is a direct application of the boundary-counting formula. The goal is to compute the sum of minimums (or maximums) over all subarrays efficiently.

For each index i , we define:

- L : the nearest index on the left where $a[L] < a[i]$ (strictly smaller for minimum)
- R : the nearest index on the right where $a[R] \leq a[i]$ (smaller or equal, to avoid double counting)

Then the contribution of $a[i]$ as the minimum of a subarray is:

$$a[i] \cdot (i - L) \cdot (R - i)$$

The total answer is:

$$\sum_{i=0}^{n-1} a[i] \cdot (i - L_i) \cdot (R_i - i)$$

Why Monotonic Stack?

Finding the nearest smaller element on both sides for all indices can be done in $O(n)$ using a monotonic stack. The stack maintains indices in increasing order of their values. When a new element breaks the order, we pop elements and record their right boundary.

This makes the entire algorithm run in $O(n)$.

Sum of Subarray Maximums

For maximums, we flip the inequality:

- L : nearest index where $a[L] > a[i]$
- R : nearest index where $a[R] \geq a[i]$

The contribution formula remains:

$$a[i] \cdot (i - L) \cdot (R - i)$$

Key Observation

The strict/non-strict distinction between left and right boundaries is important to avoid double counting when equal elements exist. A common convention is:

- left boundary: strictly smaller (or strictly greater)
- right boundary: smaller-or-equal (or greater-or-equal)

This ensures each subarray is counted exactly once.

6. Distinct-Element Contribution

The contribution technique for distinct elements in all subarrays is a clean application of boundary-counting combined with a clever change in perspective. Instead of directly counting how many distinct values appear in each subarray $[l, r]$, which would require checking all subarrays and leads to an $O(n^2)$ solution, we reverse the viewpoint and ask: how many subarrays is each occurrence responsible for as a “representative” of its value?

We start by assigning responsibility for each value to a specific occurrence: the first occurrence of that value inside a subarray. For any subarray, every distinct value contributes exactly once through its first appearance. For example, in $[1, 2, 1, 3]$, the distinct elements are $\{1, 2, 3\}$, and each is counted exactly once through its first valid occurrence in that subarray.

So instead of counting distinct values per subarray, we count how many subarrays make index i the first occurrence of its value.

To formalize this, define $\text{prev}(i)$ as the previous occurrence index of $a[i]$, or -1 if it does not exist.

For i to be the first occurrence of its value inside a subarray $[l, r]$, two conditions must hold:

- the subarray must include i , so $l \leq i \leq r$
- it must not include the previous occurrence, so $l > \text{prev}(i)$

Combining these:

$$\text{prev}(i) + 1 \leq l \leq i$$

So the number of valid choices for the left endpoint is:

$$i - \text{prev}(i)$$

For the right endpoint, there is no restriction other than containing i , so:

$$i \leq r \leq n - 1$$

which gives:

$$n - i$$

Since left and right choices are independent, the number of subarrays where index i is the first occurrence of its value is:

$$(i - \text{prev}(i))(n - i)$$

Therefore, the total number of distinct elements across all subarrays becomes:

$$\sum_{i=0}^{n-1} (i - \text{prev}(i))(n - i)$$

This is exactly the same structure as the boundary-counting formula:

$$(i - L)(R - i)$$

Here, $\text{prev}(i)$ acts as the left breaking boundary. If a subarray extends beyond $\text{prev}(i)$, then an earlier occurrence of the same value enters the subarray, and $a[i]$ is no longer the first occurrence. So the “valid region” for i is fully controlled by this boundary.

This can be computed efficiently using a single left-to-right scan with a hash map storing the last seen position of each value. For each index i , we retrieve $\text{prev}(i)$, compute its contribution $(i - \text{prev}(i))(n - i)$, and update the map. This gives an overall complexity of $O(n)$.

There is also a symmetric formulation using next occurrences instead of previous ones. Define $\text{next}(i)$ as the next occurrence of the same value. Then we can treat i as the last occurrence of its value in a subarray. In this case, valid subarrays satisfy:

- left endpoint: $0 \leq l \leq i \rightarrow i + 1$ choices

- right endpoint: $i \leq r < \text{next}(i) \rightarrow \text{next}(i) - i$ choices

So contribution becomes:

$$(i + 1)(\text{next}(i) - i)$$

Both approaches give the same final result. However, it is crucial to use only one direction (either previous or next) consistently—mixing both leads to double counting.

Final Insight

Distinct-element contribution is essentially boundary counting in disguise. The “breaking boundary” is not a smaller or greater value, but the previous (or next) occurrence of the same value. Once this boundary is identified, every index behaves like a standard contribution problem, and the total answer reduces to:

$$(i - L)(R - i)$$

applied to occurrence-based boundaries instead of value-based boundaries.

7. Bitwise Contribution (XOR / AND / OR)

Many problems ask us to compute sums involving bitwise operations such as XOR, AND, or OR over all pairs or all subarrays. At first glance, these operations look complicated because they work on entire integers. However, the key idea is simple: bitwise operations act independently on each bit position. The 0-th bit never affects the 1-st bit, and so on. This allows us to break every number into independent binary layers.

Bit Decomposition Idea

Assume each number has at most B bits. Then any number A can be written as:

$$A = \sum_{b=0}^{B-1} \text{bit}_b(A) \cdot 2^b$$

where:

- $\text{bit}_b(A) \in \{0, 1\}$
- each bit contributes independently to the final value

So instead of working with full integers, we analyze each bit separately.

Master Formula (Core Principle)

For any bitwise problem:

$$\text{Answer} = \sum_{b=0}^{B-1} 2^b \times (\text{number of structures where bit } b \text{ contributes as 1})$$

This is the most important idea in bitwise contribution:

- isolate one bit
- count how often it becomes 1
- multiply by 2^b
- sum over all bits

1. XOR Contribution

Bit behavior

A XOR bit is 1 only when the two bits are different.

So for a fixed bit b , suppose:

- cnt_0 : numbers with bit $b = 0$
- cnt_1 : numbers with bit $b = 1$

A valid pair must choose one element from each group.

So number of contributing pairs:

$$\text{cnt}_0 \times \text{cnt}_1$$

Contribution of bit b

$$\text{Contribution}_b = 2^b \cdot \text{cnt}_0 \cdot \text{cnt}_1$$

Final answer

$$\sum_{b=0}^{B-1} 2^b \cdot \text{cnt}_0(b) \cdot \text{cnt}_1(b)$$

Complexity

$$O(n \log V)$$

2. AND Contribution

Bit behavior

A bit in AND becomes 1 only if all elements have that bit = 1.

So we focus only on subarrays where the bit is continuously 1. This creates maximal runs of 1s.

Suppose a run of 1s has length L . Number of subarrays inside this run is:

$$\frac{L(L+1)}{2}$$

Contribution of bit b

$$\text{Contribution}_b = 2^b \cdot \sum_{\text{runs}} \frac{L(L+1)}{2}$$

3. OR Contribution

Bit behavior

A bit in OR becomes 1 if at least one element has bit 1.

So instead of counting where OR = 1, we count the complement. A bit is 0 only when all elements are 0.

So we again use runs of zeros. Suppose zero-runs have lengths $L_1, L_2, \dots$

Total subarrays:

$$\frac{n(n+1)}{2}$$

Subarrays where OR bit is 0:

$$\sum \frac{L_i(L_i+1)}{2}$$

So OR contribution becomes:

$$\text{Contribution}_b = 2^b \left(\frac{n(n+1)}{2} - \sum \frac{L_i(L_i+1)}{2} \right)$$

Core Insight

The entire technique reduces to:

- process one bit at a time
- convert problem into counting subarrays or pairs for that bit
- use simple combinatorics (pairs or runs)
- multiply by 2^b
- sum over all bits

Related Problems

- Sum of XOR of All Pairs
- Sum of XOR of All Subarrays
- Sum of AND of All Subarrays
- Sum of OR of All Subarrays
- Total Hamming Distance

- Pairwise Bit Difference
- XOR Beauty
- Maximum XOR Pair
- Bitwise Contribution Problems
- Prefix XOR Contribution

8. Prefix-Sum Contribution (Prefix-Pair Contribution)

Prefix-sum contribution is one of the most fundamental ideas in competitive programming because it converts a subarray problem into a pair problem on prefix indices. Instead of thinking in terms of intervals (l, r) , we think in terms of prefix positions. Every subarray corresponds to exactly one pair of prefix indices, and this is the key insight that unlocks many problems.

1. Prefix Definition and Core Transformation

We define the prefix array:

$$P[0] = 0, \quad P[i] = a_0 + a_1 + \cdots + a_{i-1}$$

Then a subarray sum becomes:

$$\text{sum}(l, r) = P[r + 1] - P[l]$$

This is extremely important because it shows that a subarray is no longer an interval—it is a pair of prefix indices $(l, r + 1)$. So instead of subarrays, we now work with pairs of prefix values.

2. Counting Subarrays with Sum = k

We want:

$$P[r] - P[l] = k$$

Rearrange:

$$P[l] = P[r] - k$$

Now we scan from left to right. For each $P[r]$, every previous prefix equal to $P[r] - k$ forms a valid subarray.

So we maintain a frequency map:

$$\text{freq}[P[r] - k]$$

Each occurrence directly contributes to the answer.

So a subarray problem becomes a running pair-counting problem on prefix values.

3. Subarrays Divisible by k

A subarray sum is divisible by k if:

$$P[r] - P[l] \equiv 0 \pmod{k}$$

So:

$$P[r] \equiv P[l] \pmod{k}$$

This means:

- valid subarrays come from equal prefix remainders

If a remainder class has size m , then number of valid pairs is:

$$\binom{m}{2} = \frac{m(m-1)}{2}$$

So final answer is:

$$\sum_{\text{remainder}} \binom{\text{cnt}[\text{remainder}]}{2}$$

Again, we never look at subarrays directly—we only count pairs of prefix indices.

4. Sum of All Subarray Sums (Prefix Contribution View)

Every subarray sum is:

$$P[r] - P[l]$$

So:

$$\sum_{\text{subarrays}} \text{sum} = \sum_{0 \leq l < r \leq n} (P[r] - P[l])$$

Split it into two parts:

$$= \sum_{0 \leq l < r \leq n} P[r] - \sum_{0 \leq l < r \leq n} P[l]$$

Contribution of $P[r]$

For a fixed r , it appears once for every $l < r$, so r times.

So:

$$\sum_{0 \leq l < r \leq n} P[r] = \sum_{r=0}^n P[r] \cdot r$$

Contribution of $P[l]$

For a fixed l , it appears once for every $r > l$, so $(n - l)$ times.

So:

$$\sum_{0 \leq l < r \leq n} P[l] = \sum_{l=0}^n P[l] \cdot (n - l)$$

5. Final Combined Formula

$$\sum_{\text{subarrays}} \text{sum} = \sum_{k=0}^n P[k] \cdot k - \sum_{k=0}^n P[k] \cdot (n - k)$$

Merge terms:

$$= \sum_{k=0}^n P[k] (2k - n)$$

6. Meaning of the Coefficient $2k - n$

Each prefix $P[k]$ contributes in two ways:

- As a right endpoint: contributes positively $\rightarrow$ appears k times
- As a left endpoint: contributes negatively $\rightarrow$ appears $n - k$ times

So net contribution:

$$k - (n - k) = 2k - n$$

This is pure contribution counting:

- positive appearances minus negative appearances

7. Core Insight

Prefix-sum contribution shows that:

- every subarray = pair of prefix indices
- every condition on subarray = condition on prefix pairs
- every subarray sum = difference of two prefix values

So we convert:

subarray problem $\rightarrow$ pair problem on prefix array

8. Final Unifying Principle

If a subarray expression can be written as:

$$H(P[r], P[l])$$

then we should stop thinking about subarrays and instead think about:

pairs of prefix indices

9. Problem Types Covered by This Idea

This framework solves:

- Subarray sum = k
- Subarray sum divisible by k
- Sum of all subarray sums
- Subarray XOR problems (using prefix XOR)
- Equal 0/1 subarray counts
- Target difference subarray problems
- Frequency-based subarray constraints

Final Intuition

Prefix-sum contribution is just pair contribution on prefix arrays.

Once this is understood, almost every subarray sum problem becomes:

- build prefix array
- count valid prefix pairs
- apply contribution logic

That is the entire technique in one idea.

9. Median Contribution (Balance Transformation + Prefix Pair View)

Median-related subarray problems look significantly harder than sum, minimum, maximum, or XOR problems because the median is an order statistic, not an algebraic one. To compute a median directly, we would normally need to sort each subarray or find its middle element after ordering. Doing this for every subarray is infeasible for large n .

The key breakthrough is the **balance transformation**, which converts the median condition into a simple prefix-sum contribution problem.

1. Reducing Median to a Threshold Problem

Instead of directly computing medians, we ask a simpler question:

Is the median of a subarray at least x ?

Now we only care about whether elements are above or below a threshold x , not their actual values.

We transform the array into:

$$b[i] = \begin{cases} +1 & \text{if } a[i] \geq x \\ -1 & \text{if } a[i] < x \end{cases}$$

This transformation discards actual values but preserves exactly what we need for median comparison.

2. Why This Works

Consider an odd-length subarray. Its median is at least x if and only if more than half of its elements are $\geq x$.

In the transformed array:

- $+1$ represents elements $\geq x$
- -1 represents elements $< x$

So the subarray sum becomes:

$$(\# \geq x) - (\# < x)$$

Thus:

- positive sum $\Rightarrow$ more elements $\geq x \Rightarrow$ median $\geq x$
- non-positive sum $\Rightarrow$ median $< x$

So:

$$\text{median}(a[l..r]) \geq x \iff \sum b[i] > 0$$

3. Prefix Sum Conversion

Define prefix sums of the transformed array:

$$B[0] = 0, \quad B[i] = b[0] + b[1] + \dots + b[i-1]$$

Then any subarray sum becomes:

$$\sum_{k=l}^r b[k] = B[r+1] - B[l]$$

So the median condition becomes:

$$B[r+1] - B[l] > 0$$

Rearranging:

$$B[r+1] > B[l]$$

4. Key Insight: Median $\rightarrow$ Prefix Pair Problem

Now the problem is no longer about medians at all. It becomes:

Count pairs of prefix indices $(l, r+1)$ such that $B[l] < B[r+1]$

So a median problem has been fully converted into a **pair-contribution problem on prefix sums**.

5. Connection to Previous Contribution Ideas

This is structurally identical to earlier prefix techniques:

- $\text{Sum} = k \Rightarrow P[r] - P[l] = k$
- Modulo condition $\Rightarrow P[r] \equiv P[l]$
- Now median $\Rightarrow B[l] < B[r]$

In all cases:

- subarrays disappear
- only prefix pairs remain
- we count relationships between prefix values

6. Efficient Counting (Contribution Style)

We process prefix sums from left to right. For each $B[r]$, we need:

how many previous prefix values are strictly smaller than it

This is a classic **inversion-style counting** problem. We maintain:

- Fenwick Tree / BIT
- Segment Tree
- or ordered multiset

Procedure:

1. Query how many previous values $< B[r]$
2. Add that count to answer
3. Insert $B[r]$ into the structure

Each operation takes $O(\log n)$, so total complexity is:

$$O(n \log n)$$

7. When x is Unknown (Binary Search Trick)

Often we want:

- median of subarrays
- k -th smallest median
- threshold-based median conditions

Define:

$$f(x) = \text{number of subarrays with median} \geq x$$

Key property: as x increases, $f(x)$ decreases. So $f(x)$ is monotonic:

$$x_1 < x_2 \Rightarrow f(x_1) \geq f(x_2)$$

This allows **binary search** on x .

8. Full Checking Procedure

For a candidate x :

1. Build $b[i]$ as $+1 / -1$
2. Build prefix array $B[i]$
3. Count pairs $B[l] < B[r]$
4. Get $f(x)$
5. Adjust binary search

Total complexity:

$$O(n \log n \log V)$$

9. Final Insight

Median problems, despite appearing fundamentally different, reduce to:

- a threshold transformation
- a prefix sum construction
- a pair counting condition

So the entire idea becomes:

Median $\rightarrow$ Balance array $\rightarrow$ Prefix sums $\rightarrow$ Count prefix pairs

Final Mental Model

Instead of thinking: *“how do I find the median?”*

We think: *“how many prefix pairs satisfy a simple inequality?”*

That is the core of the **median contribution technique**—it turns an order-statistic problem into a standard prefix-pair counting problem, exactly like all other contribution frameworks.

10. GCD / LCM Contribution (Subarray Group Compression Technique)

GCD contribution problems look difficult at first because there are $O(n^2)$ subarrays, and computing the GCD of each subarray individually would be too slow. The key observation that makes these problems efficient is that the GCD of a subarray evolves in a very structured way when we extend the subarray.

1. Key Behavior: GCD is Monotonic

Fix a left endpoint l , and gradually extend the right endpoint r . We observe:

$$\gcd(a[l..r])$$

can never increase. Every time we add a new element:

- the GCD either stays the same
- or decreases

So GCD follows a **monotonic non-increasing** behavior.

2. Key Structural Insight: Limited Number of Changes

The deeper reason this works is that the GCD cannot change too many times. If the current GCD is d , and it changes to d' , then:

- d' must be a divisor of d
- so d' is strictly smaller

A crucial property is:

$$d' \leq \frac{d}{2}$$

So whenever GCD changes, it drops by at least a factor of 2. Starting from at most V , this means the GCD can only change about:

$$O(\log V)$$

times. So even though there are $O(n)$ extensions, there are only $O(\log V)$ distinct GCD values for a fixed starting index.

3. Grouping Idea (Compression of Right Endpoints)

Instead of tracking GCD for every r , we compress them into groups. Example:

$$60, 60, 60, 30, 30, 10, 10, 5, 1, 1, \dots$$

We rewrite this as:

- $(60, [r_1, r_2])$
- $(30, [r_3, r_4])$
- $(10, [r_5, r_6])$
- $(5, [r_7, r_7])$
- $(1, [r_8, n - 1])$

Each group represents:

- a range of right endpoints
- where the GCD stays constant

So for a fixed l , all subarrays are compressed into only $O(\log V)$ groups.

4. Contribution Formulation

Now suppose we want:

$$\sum_{\text{subarrays}} \gcd(\text{subarray})$$

For a group:

- GCD value = g
- number of right endpoints = m

Then all those subarrays contribute:

$$g \cdot m$$

So total contribution for a fixed l is:

$$\sum_{\text{groups}} g \cdot m$$

Summing over all l gives the final answer.

5. Dynamic Construction (Scan from Left to Right)

Instead of recomputing from scratch:

- we maintain the list of GCD groups ending at position $i - 1$
- when we add $a[i]$, we update every group:

$$g \rightarrow \gcd(g, a[i])$$

Different groups may merge if they become equal. Since there are only $O(\log V)$ groups at any time:

- each position costs $O(\log V)$
- total complexity:

$$O(n \log V)$$

6. Counting Subarrays with GCD = k

This framework also solves:

- count subarrays where $\gcd = k$

Instead of summing values:

- we just count groups where $g = k$

So each group directly contributes its size to the answer.

7. Core Interpretation

This is still a contribution technique, but instead of:

- index contribution
- bit contribution
- boundary contribution

we are doing: **contribution of GCD groups**.

Each group represents a block of subarrays sharing the same GCD, so we aggregate them at once.

8. Why LCM Does NOT Behave Similarly

LCM looks similar but behaves very differently. When extending a subarray:

- LCM can only stay same or increase

However:

- it does **NOT** have a logarithmic bound on changes

For example:

$$2, 3, 5, 7, 11, 13, \dots$$

LCM evolves like:

$$2 \rightarrow 6 \rightarrow 30 \rightarrow 210 \rightarrow 2310 \rightarrow \dots$$

Each new prime introduces a new factor, so:

- number of distinct LCM values can grow linearly

So unlike GCD:

- no $O(\log V)$ compression exists in general

9. Practical LCM Behavior

LCM problems usually work only under constraints like:

- LCM must stay $\leq K$
- or early termination is possible

Because once LCM grows too large:

- extending subarray becomes useless

So LCM solutions are often **constraint-driven**, not structure-driven like GCD.

10. Final Insight

GCD is special because:

- it only decreases
- and decreases logarithmically
- allowing group compression

So every subarray starting at l collapses into:

$O(\log V)$ GCD groups

Final Mental Model

Whenever you see:

- $\sum \text{gcd}(\text{subarray})$
- count subarrays with $\text{gcd} = k$
- number of distinct gcd values

You should immediately think:

maintain compressed GCD groups for each starting position and aggregate contributions per group

This is the core contribution technique behind almost all advanced GCD subarray problems.

11. Frequency-Based Contribution (Occurrence Contribution)

Frequency-based contribution problems arise whenever a condition depends on how many times a value appears inside a subarray rather than on the values themselves. Examples include counting subarrays containing exactly K occurrences of a number, at least K occurrences, at most K occurrences, or treating an occurrence as the first, last, or K -th occurrence within a subarray. Although these problems initially look different from contribution techniques based on minimums, maximums, or distinct elements, they ultimately reduce to the same boundary-counting philosophy.

1. The Exactly- K Trick

One of the most important identities in this area is the **exactly- K trick**. Conditions involving “exactly K ” occurrences are usually difficult because they are not monotonic. If a subarray has exactly K occurrences and we extend it, it may no longer satisfy the condition. However, “at least K ” occurrences is monotonic: once a subarray contains at least K occurrences, extending it cannot destroy that property. This leads to the identity:

$$\text{exactly}(K) = \text{atLeast}(K) - \text{atLeast}(K + 1)$$

The idea is simple. Every subarray counted by $\text{atLeast}(K)$ contains K or more occurrences. Among them, those with $K + 1$ or more occurrences are precisely the ones counted by $\text{atLeast}(K + 1)$. Subtracting removes all subarrays with more than K occurrences, leaving only those with exactly K . This transformation is extremely powerful because it converts a non-monotonic condition into two monotonic counting problems that can often be solved with sliding windows or contribution techniques.

2. Two-Pointer Method for $\text{atLeast}(K)$

A common way to count $\text{atLeast}(K)$ occurrences is with the **two-pointer method**. Suppose we want to count subarrays containing at least K occurrences of a value v . As we move the right endpoint r from left to right, we maintain the smallest left endpoint $l^*(r)$ such that the subarray $[l^*(r), r]$ still contains at least K occurrences of v . Because removing elements from the left can only decrease the occurrence count, this boundary never moves left; it can only stay or move right. Thus $l^*(r)$ is monotonic, making a two-pointer sweep possible.

Once $l^*(r)$ is known, every left endpoint $0, 1, 2, \dots, l^*(r)$ produces a valid subarray ending at r . Therefore the number of valid subarrays ending at r is:

$$l^*(r) + 1$$

Summing this quantity over all right endpoints gives the total count. Since both pointers move at most n times, the entire process runs in $O(n)$.

3. Contribution via Occurrence Positions

Although the sliding-window approach is often the most practical, there is also a pure contribution-style derivation based on occurrence positions. Suppose the value v appears at positions:

$$p_1 < p_2 < \dots < p_m$$

A subarray contains at least K occurrences of v if it contains some block of K consecutive occurrences $p_i, p_{i+1}, \dots, p_{i+K-1}$. To avoid double counting, we assign each valid subarray to its **leftmost qualifying block** of K occurrences. This is exactly the same responsibility-assignment idea used for minimums, maximums, and distinct elements.

4. Boundary Formula for a Block

Consider the block $p_i, \dots, p_{i+K-1}$. Define:

$$L_b = p_{i-1} \quad (\text{or } -1 \text{ if no previous occurrence})$$

$$R_b = p_{i+K} \quad (\text{or } n \text{ if no next occurrence})$$

For this block to be the leftmost block of K occurrences inside a subarray:

- the left endpoint must lie after L_b but no later than p_i , giving $p_i - L_b$ choices
- the right endpoint must include p_{i+K-1} but stop before R_b , giving $R_b - p_{i+K-1}$ choices

Since the left and right choices are independent, the contribution of this block is:

$$(p_i - L_b) \cdot (R_b - p_{i+K-1})$$

Notice how familiar this formula looks. It is exactly the same boundary-counting pattern:

$$(i - L)(R - i)$$

that appeared in minimum contribution, maximum contribution, and distinct-element contribution. The only difference is that the boundaries are now defined by **neighboring occurrences** of the same value rather than by smaller or larger elements.

5. Counting Subarrays Where p_i is the K -th Occurrence

This viewpoint becomes even more useful when a problem asks about the K -th occurrence itself. Suppose p_i is an occurrence of value v , and we want to count subarrays where p_i is the K -th occurrence of v inside the subarray. Then the subarray must contain exactly the occurrence window:

$$p_{i-K+1}, \dots, p_i$$

The left endpoint must be chosen after the occurrence immediately before this window, and the right endpoint must be chosen before the occurrence immediately after it. Once again, the number of valid subarrays becomes a simple **product of a left-choice count and a right-choice count**. The derivation is identical to the one used for at-least- K occurrences.

6. Core Interpretation

The most important insight is that frequency-based contribution problems are **not a separate category**. They belong to the same family as minimum contribution, maximum contribution, and distinct-element contribution. In all of these problems:

- an index or a block of occurrences is designated as the **special representative** for a subarray
- the subarray remains valid only while its endpoints stay between certain **breaking boundaries**
- once those boundaries are identified, the count always takes the familiar form:

$$(\text{left choices}) \times (\text{right choices})$$

7. Final Insight

The same universal boundary-counting formula:

$$(i - L)(R - i)$$

reappears again and again. The meaning of L and R changes from problem to problem:

- sometimes they are previous smaller elements
- sometimes previous occurrences
- sometimes neighboring occurrence blocks

But the counting mechanism remains exactly the same. This recurring pattern is one of the strongest signals that a contribution-based solution exists.

Final Mental Model

Whenever you see:

- count subarrays with exactly K occurrences of v
- count subarrays with at least K occurrences of v
- count subarrays where p_i is the K -th occurrence

You should immediately think:

assign each subarray to a representative block, find the breaking boundaries, and multiply left choices by right choices

12. Structural Contribution (Tree / Graph Contribution)

Your previous contribution techniques (single element, pair, bitwise, boundary, prefix, GCD, etc.) all share one common idea: they define what is being counted inside a structure.

But notice something important:

A contribution technique is not tied to arrays — it is tied to how we choose independent units.

A tree or graph is just another structure, like an array. So instead of thinking “tree contribution is a new technique”, we should understand it as:

The same contribution framework applied to a different structure of objects.

1. Core Philosophy Shift

In array problems, the independent contributors are:

- elements
- pairs
- bits
- boundaries
- prefix values

In tree/graph problems, the independent contributors become:

- nodes
- edges
- paths
- subtrees
- ancestors
- depths

So the idea is unchanged. Only the **unit of contribution** changes.

2. Universal Contribution View

Instead of writing:

$$\sum_{\text{subarrays}} f(\text{subarray}) \quad \text{or} \quad \sum_{\text{pairs}} f(i, j)$$

in trees we typically see:

$$\sum_{\text{paths}} f(\text{path}) \quad \text{or} \quad \sum_{\text{subtrees}} f(\text{subtree}) \quad \text{or} \quad \sum_{\text{pairs of nodes}} f(u, v)$$

The key step is always: **swap summation** → move from structures to units. So:

$$f = \sum \text{contribution of basic unit}$$

3. Node Contribution

Suppose every path contributes node values. Instead of enumerating all paths, we fix one node u .

Let:

$$\text{cnt}(u) = \text{number of paths that contain } u$$

Then:

$$\text{Contribution}(u) = \text{value}(u) \cdot \text{cnt}(u)$$

So the total answer becomes:

$$\sum_u \text{value}(u) \cdot \text{cnt}(u)$$

Derivation Insight. Start from:

$$\sum_{\text{paths}} \sum_{u \in \text{path}} \text{value}(u)$$

Swap summations:

$$= \sum_u \text{value}(u) \cdot (\text{number of paths containing } u)$$

So again: **count** $\times$ **value**. Exactly like $a[i](i+1)(n-i)$ in arrays.

4. Edge Contribution

Now consider pair problems like:

$$\sum_{u < v} \text{distance}(u, v)$$

Instead of paths, fix an edge. Removing an edge splits the tree into two parts:

- size s
- size $n - s$

Any path crossing this edge must pick one node from the left part and one from the right. So:

$$\text{cnt}(e) = s(n - s)$$

If edge has weight w_e :

$$\text{Contribution}(e) = w_e \cdot s(n - s)$$

Thus:

$$\sum_{\text{edges}} w_e \cdot s(n - s)$$

5. Why This Works

Every pair (u, v) has exactly one path between them. So instead of counting paths:

we count how many pairs force an edge to be used

That transforms:

$$\text{global structure (all paths)} \longrightarrow \text{local structure (one edge split)}$$

6. Subtree Contribution

Many problems ask about subtree size, subtree sum, or contributions over all subtrees. Instead of enumerating all subtrees, we fix a node. Each node contributes to all subtrees that include it. So again:

$$\text{Contribution}(\text{node}) = \text{value}(\text{node}) \cdot \text{cnt}(\text{node})$$

where $\text{cnt}(\text{node})$ is the number of subtrees containing this node. This is again a **counting problem disguised as structure enumeration**.

7. Path Contribution View

For path-based problems:

$$\sum_{\text{paths}} f(\text{path})$$

We always try: fix a node or edge, then count how many paths pass through it. Then:

$$\text{Contribution}(\text{unit}) = \text{value}(\text{unit}) \cdot \text{number of paths using it}$$

This is identical in spirit to **boundary contribution** in arrays.

8. Rerooting Contribution

Sometimes contribution depends on root choice. Instead of recomputing from scratch, we transfer DP from parent $\rightarrow$ child. But the idea stays the same:

- the unit of contribution does not change
- only the count changes with root

So rerooting is just: **dynamic re-evaluation of contribution counts**.

9. General Structural Formula

Almost all tree/graph contribution problems reduce to:

$$\text{Contribution}(\text{unit}) = \text{Value}(\text{unit}) \cdot \text{Count}(\text{unit})$$

Where the unit can be: node, edge, subtree, path, ancestor, or depth.

And the final answer:

$$\sum \text{Contribution}(\text{unit})$$

10. Recognition Pattern

You should think of structural contribution when you see:

- sum over all paths
- sum over all node pairs
- subtree aggregation
- edge-based contributions
- rerooting DP
- “every node affects many paths”

Final Mental Trigger

The key question is always:

Instead of enumerating all paths/subtrees, can I fix one unit (node/edge) and count how many structures contain it?

If yes, then:

you are no longer doing graph theory — you are doing contribution technique on a structure.